

AI Agent-Driven Development with Formal Methods — The Human Role in the Era of Multi-Agent Coordination

Hikaru Ando (安藤光太郎) — IID Systems

— *Toward a Near Future Where AI Agent Teams Autonomously Build Systems Using Formal Specifications as Contracts* —

Introduction: Entering the Era of AI Agent-Driven Development

The rapid evolution of large language models (LLMs) is fundamentally challenging how software is built. AI tools such as GitHub Copilot, Claude, and GPT-4 already demonstrate practical capability in code generation, review, and refactoring. But the trajectory points beyond “using AI as a tool”—toward a new paradigm in which **multiple AI agents coordinate like a development team, autonomously constructing systems**.

This raises a fundamental question: when multiple AI agents coordinate, what serves as the “team’s common language”? Vague natural-language specifications cannot eliminate interpretive discrepancies between agents. Applying Test-Driven Development (TDD) doesn’t help either—tests are grounded in inductive reasoning and cannot function as a mechanism for inter-agent specification agreement.

What this article proposes is a multi-agent development paradigm in which formal methods—specifically VDM (Vienna Development Method)—serve as the backbone of specification, with formal specifications functioning as **rigorous contracts between agents**. Think of it like building a house: the homeowner (human) says “I want a bright, open living room with lots of natural light.” The builder (AI) considers the budget, climate, and site conditions, then proposes a design—perhaps a vaulted ceiling with floor-to-ceiling windows. The sequencing of foundation, framing, electrical, and plumbing is not the homeowner’s concern; that’s the builder’s expertise. Each subcontractor (AI agent) understands its boundaries through the formal specification—the “blueprint”—enabling unambiguous coordination.

This article first identifies the fundamental limitations of TDD (Chapter 1), then argues why formal methods are ideally suited as “contracts” between AI agents (Chapters 2–5), presents a concrete multi-agent coordination architecture (Section 7.3), and discusses **how the human role is re-defined** in this new paradigm—as a domain expert who defines requirements, while technical design and implementation are delegated to AI.

Chapter 1: Why Test-Driven Development Is Fundamentally Flawed

Before proceeding, a clarification: this article critiques the *paradigm* of TDD—the idea that tests should be the central driver of design and quality assurance—not the act of testing itself. The role testing continues to play is addressed in Section 7.2.

1.1 The Fundamental Limit of Tests: The Trap of Inductive Reasoning

The core problem with TDD is that tests are grounded in **inductive reasoning**.

A test asks: “For this finite set of inputs, does the program produce the expected output?” But the input space of virtually any real program is infinite or astronomically large. Edsger W. Dijkstra put it plainly:

“Program testing can be used to show the presence of bugs, but never to show their absence.” — Edsger W. Dijkstra, 1969

This is not merely a pithy observation—it is a logically precise statement. Even if every test case $\{t_1, t_2, \dots, t_n\}$ passes, there is no guarantee that the program behaves correctly for some untested input t_{n+1} . This is the problem of incomplete induction.

1.2 The Mathematics of Incompleteness

Consider an integer addition function `add(a, b)`. If both `a` and `b` are 32-bit integers, the number of possible input pairs is $2^{32} \times 2^{32} = 2^{64} \approx 1.8 \times 10^{19}$. Running one test per nano-second, full coverage would take approximately 585 years.

In real systems, inputs are far more complex than integer pairs. API requests, database states, timing, concurrency ordering—the state space is effectively infinite. Every TDD test suite is sampling an infinitesimally small region of that space.

1.3 The Counterargument That TDD Is a Design Method

TDD advocates often reframe it: “TDD is not a testing method—it is a design method.” Writing tests first forces you to design testable interfaces, the argument goes.

But this argument has a structural problem. When test-writability becomes the criterion for good design, the tail wags the dog. A testable interface is not necessarily a good interface. Design should be derived from the **logical structure of requirements**, not reverse-engineered from the convenience of tests.

In formal methods, the specification itself drives design. By explicitly stating invariants, pre-conditions, and post-conditions, interface correctness is guaranteed at the specification level. There is no need to entrust design to the accidental selection of test cases.

1.4 The Illusion of Coverage

100% code coverage is often cited as a quality metric. But coverage measures which *lines of code were executed*, not which *parts of the specification were verified*.

Consider this function:

```
def divide(a: int, b: int) -> float:
    return a / b
```

The test `divide(10, 2) == 5.0` achieves 100% line coverage. But the case `b = 0` is never tested. Coverage is fundamentally insufficient as a measure of specification completeness.

A formal specification writes it differently:

```
divide(a: int, b: int) -> float
  pre: b ≠ 0
  post: result * b = a
```

The pre-condition `b ≠ 0` resolves the division-by-zero problem at the specification level. No test case can be forgotten because the constraint is structural, not incidental.

Chapter 2: What Are Formal Methods? A Focus on VDM

2.1 The Core Idea

Formal methods are a family of techniques that use mathematical notation and inference rules to describe software specifications and reason about their correctness.

Major formal methods include:

- **VDM (Vienna Development Method):** Developed at IBM's Vienna lab in the 1970s. Features the model-oriented specification language VDM-SL. Extensive industrial application history.
- **Z Notation:** From Oxford University. Specification based on set theory and schemas.
- **B Method:** Used in systems such as the Paris Métro automatic train operation. High affinity with automated proof.
- **Alloy:** A lightweight formal method from MIT. Supports automatic verification through model checking.
- **TLA+:** Developed by Leslie Lamport. Particularly strong for distributed systems. Adopted extensively by Amazon.

This article focuses on VDM for three reasons: VDM-SL is well-suited for AI generation and manipulation; it integrates well with mechanical consistency checking via tools like Overture Tool; and it has a strong industrial track record.

2.2 Core Elements of VDM-SL

Type Definitions: VDM-SL defines the structure of data as types. Beyond primitive types (`nat`, `int`, `bool`, `char`), it supports set types (`set of T`), sequence types (`seq of T`), map types (`map T1 to T2`), product types, and union types.

Invariants: Conditions that a type or system state must always satisfy, expressed in predicate logic. This ensures that invalid states cannot exist at the specification level.

Operations: State-changing operations defined as pairs of pre-conditions and post-conditions. “What must hold before this operation is called?” and “What must hold after it completes?” are made explicit.

Functions: Pure computations without side effects. In implicit specifications, only the relationship between inputs and outputs is stated as a predicate—no concrete algorithm is specified.

2.3 Deductive vs. Inductive Reasoning

The decisive difference between TDD and formal methods lies in the direction of reasoning.

TDD (inductive) infers general correctness from a finite set of specific test cases. Formal specification (deductive) defines an unambiguous criterion for correctness, from which any conformant implementation can be reasoned about.

An important distinction must be made here. Formal specification and formal verification are separate processes. Writing a VDM-SL specification guarantees the **internal consistency** (freedom from contradiction) and **completeness** (all required operations are defined) of the specification itself. Verifying that an implementation *conforms* to the specification requires separate means—theorem provers, property-based testing, etc.—addressed in Chapter 4, Phase 4.

Nevertheless, inductive reasoning has a fundamental ceiling: no accumulation of test cases reaches certainty. The formal specification approach at minimum defines unambiguously what “correct” means, enabling verification against a clear standard. This is a qualitative difference from TDD.

Chapter 3: VDM-SL in Practice — A Concrete Example

3.1 Example: A User Management System

A note before the code: this article argues that humans do not need to read formal specifications. So why show VDM-SL here at all? For the same reason a building client benefits from knowing that structural calculations exist and what role they play—even without reading them. The following example illustrates how rigorous, and how complete, the artifact generated by AI actually is.

```

-- Formal specification of a user management system

types
  UserId = nat1
  inv uid == uid <= 999999;

  Email = seq1 of char
  inv email ==
    exists i in set inds email & email(i) = '@'
    and len email <= 254;

  Password = seq of char
  inv pw == len pw >= 8 and len pw <= 128;

  Role = <Admin> | <Editor> | <Viewer>;

  User :: id      : UserId
        email    : Email
        name     : seq1 of char
        role     : Role
        active   : bool
  inv u == len u.name <= 100;

state UserSystem of
  users    : map UserId to User
  nextId   : UserId
  emails   : set of Email
  inv mk_UserSystem(users, nextId, emails) ==
    -- All user IDs are less than nextId
    (forall uid in set dom users & uid < nextId)
    -- emails matches the set of all registered email addresses
    and emails = {users(uid).email | uid in set dom users}
    -- Each user's id field matches its key in the map
    and (forall uid in set dom users & users(uid).id = uid)
  init s == s = mk_UserSystem({|->}, 1, {})
end

operations

RegisterUser(email: Email, name: seq1 of char, role: Role) uid: UserId
  ext wr users : map UserId to User
  wr nextId : UserId
  wr emails : set of Email
  pre email not in set emails -- email uniqueness
  and nextId <= 999999 -- ID ceiling
  post let newUser = mk_User(nextId~, email, name, role, true) in
    uid = nextId~
    and users = users~ munion {nextId~ |-> newUser}
    and nextId = nextId~ + 1
    and emails = emails~ union {email};

DeactivateUser(uid: UserId)
  ext wr users : map UserId to User

```

```

pre uid in set dom users
  and users(uid).active = true
post users = users~ ++
  {uid |-> mu(users~(uid), active |-> false)};

ChangeRole(uid: UserId, newRole: Role)
  ext wr users : map UserId to User
  pre uid in set dom users
    and users(uid).active = true
  post users = users~ ++
    {uid |-> mu(users~(uid), role |-> newRole)};

FindUserByEmail(email: Email) result: [UserId]
  ext rd users : map UserId to User
  post if exists uid in set dom users & users(uid).email = email
    then result <> nil
      and result in set dom users
      and users(result).email = email
    else result = nil;

functions

ActiveUserCount: map UserId to User -> nat
ActiveUserCount(users) ==
  card {uid | uid in set dom users & users(uid).active};

HasAdmin: map UserId to User -> bool
HasAdmin(users) ==
  exists uid in set dom users &
    users(uid).role = <Admin> and users(uid).active;

```

3.2 What This Specification Directly Guarantees

Reading the specification carefully reveals that the following properties are **logically guaranteed**—not probabilistically suggested by test cases.

1. Email uniqueness: The pre-condition `email not in set emails` in `RegisterUser` makes duplicate registration with the same address logically impossible at the specification level. Guaranteeing this with TDD would require testing every possible registration order and concurrency pattern—an intractable task.

2. State consistency: The system invariant `inv mk_UserSystem(...)` ensures that the `emails` set and `users` map remain permanently synchronized, and that ID integrity is always maintained. That post-conditions preserve this invariant can be confirmed deductively from the specification.

3. Type constraints: `UserId` is a natural number between 1 and 999999; `Email` is a string of 1–254 characters containing `@`; `Password` is 8–128 characters. These constraints are built into the type definitions.

3.3 Design Challenges Arising from Specification Contracts

Section 3.2 addressed properties that the specification directly guarantees. But formal specifications serve a second, equally important function: because their contracts are explicit, they **surface design challenges** that would otherwise remain invisible until implementation.

Pre-conditions are responsibility-boundary contracts. For example, the pre-condition `users(uid).active = true` in `ChangeRole` declares: “changing the role of an inactive user is outside this operation’s scope.” Invalid inputs do not magically disappear—the contract states: “if this condition does not hold, this operation’s behavior is not guaranteed.”

This explicit contract gives rise to two concrete design challenges.

Design challenge 1: Compositional verification across modules. If module A calls `Register-User` (post-condition: `active = true`) and module B immediately calls `ChangeRole`, A’s post-condition logically implies B’s pre-condition ($A.post \Rightarrow B.pre$). This can be formally confirmed—proving that the module composition is free of contradiction at the specification level. This is a guarantee that testing struggles to achieve. This verification is performed automatically by AI during Phase 2 (design).

Design challenge 2: Defensive placement. When data that violates a pre-condition reaches the system, the question becomes: where and how to defend? Should the validation layer sit at the API gateway? At the entry point of each module? Should a data quarantine (sanitization) module be inserted, and at which architectural layer? These are design questions, not specification questions—but they only surface as concrete agenda items *because* the pre-conditions are explicit. With ambiguous natural-language specifications, there is a real risk that these defensive design discussions never occur before implementation begins.

These design challenges are resolved concretely in Phase 2 (AI-driven design) of the workflow described in Chapter 4. Formal specifications simultaneously deliver two forms of value: **correctness guarantees** (Section 3.2) and **design challenge clarification** (this section).

Chapter 4: The AI-Driven Development Workflow

4.1 The Proposed Workflow

Phase 1: Homeowner and Builder — Dialogue-Driven Formal Specification

This phase mirrors the relationship between a homeowner and a builder. The homeowner says “I want a bright, open living room”—they don’t concern themselves with structural calculations or material specifications. The builder (AI) considers constraints such as budget, site conditions, and climate to propose a concrete design. If the homeowner lives in a cold region and wants to

minimize heating costs, the builder won't propose floor-to-ceiling glass; instead, they'll suggest alternatives that achieve openness within those constraints. Through this iterative dialogue of proposals, evaluation, and refinement, the specification is built up.

The human communicates business requirements in natural language. AI generates VDM-SL. The human never reads the formal specification directly. AI explains the specification's content and implications in natural language; the human judges whether the explanation matches their intent.

A concrete example of this dialogue: the human says, "I want to prevent duplicate email addresses at registration." AI responds: "I've defined the pre-condition `email not in set emails`. This makes re-registration with an already-registered address logically impossible. However, should email addresses from deactivated accounts be reusable? I need your decision on that." The human decides based on their domain knowledge.

In this phase, the human is responsible for:

- Providing business domain knowledge
- Hearing AI's explanations and judging specification validity — "Is that actually the intended behavior?"
- Asking about edge cases — "What happens when a deactivated user tries to re-register?"
- Final approval of the specification

AI is responsible for: converting natural-language requirements into VDM-SL; checking internal consistency; explaining the specification and its implications clearly; and surfacing potential edge cases and contradictions.

The key point: the formal specification is an artifact that AI reads and writes. Humans need not understand VDM-SL notation. Why use VDM-SL rather than a natural-language specification document? This question is addressed in Section 5.3.

Here is another concrete example—a Phase 1 dialogue for the seemingly simple requirement of "sorting numbers":

Human: I want to sort input numbers in ascending order. **AI:** So for `{3, 2, 7}`, the output should be `[2, 3, 7]`? **Human:** Yes. **AI:** Just to confirm—would `[1, 2, 3]` be acceptable for `{3, 2, 7}`? That is, is any ascending sequence fine, or must the output contain only the input elements? **Human:** It must contain all elements from the input. **AI:** Then for `{3, 3, 1, 8}`, would `[1, 3, 8]` be correct? Should duplicates be collapsed? **Human:** No—just rearrange the original values. Duplicates remain as they are.

After three rounds of dialogue, AI generates the following VDM-SL specification:

```

functions

sort[@T]: seq of @T * (@T * @T -> bool) -> seq of @T
sort(l, less) == is not yet specified
post bagOf[@T](l) = bagOf[@T](RESULT)          -- Permutation
    and forall i in set {1, ..., len RESULT - 1} &
        not less(RESULT(i+1), RESULT(i))        -- Sorted
measure len l;

bagOf[@T]: seq of @T -> map @T to nat
bagOf(s) ==
    { i |-> occurs[@T](i, s) | i in set elems s }
post dom RESULT = elems s and sizeOfBag[@T](RESULT) = len s;

occurs[@T]: @T * seq of @T -> nat
occurs(e, s) ==
    if s = [] then 0
    else (if e = hd s then 1 else 0) + occurs[@T](e, tl s)
measure len s;

```

Note what happened: three rounds of misunderstanding in natural language were resolved by two post-conditions in VDM-SL. `bagOf(l) = bagOf(RESULT)` means “the output is a permutation of the input (same elements, same counts).” The ordering condition means “no adjacent pair is out of order.” This specification is satisfied by bubble sort, quicksort, and merge sort alike. **Algorithm selection is a design decision, not a specification concern.**

Phase 1 Supplement: Formalizing Existing Specifications — The MD→VDM-SL→MD Pipeline

The Phase 1 dialogue examples above assumed building specifications from scratch. In practice, however, development teams often already have incomplete specification documents — requirements documents, design docs, API specs — written in Markdown or similar formats. In such cases, it is more efficient to start from the existing document rather than from zero.

The core insight of this approach is that **the very act of attempting to convert natural language to VDM-SL functions as a specification verification activity**. When you try to write VDM-SL type definitions, pre-conditions, or post-conditions, the “parts you cannot write” emerge structurally. The reason you cannot write them is that the original natural language specification is ambiguous.

Here is the concrete process, illustrated with an e-commerce order processing specification.

Step 1: Document Structure Analysis and Requirement Classification

AI reads the Markdown document and automatically classifies its structural elements. Headings become module or agent boundary candidates. Bullet lists map to requirements, constraints, and rules. Tables become data models. Numbered lists correspond to sequential operations or work-

flows. Each requirement is classified by which VDM-SL construct it maps to: “A product has an ID, a name, and a stock count” maps to type definitions (`types`); “Stock cannot go below zero” maps to an invariant (`inv`); “Users can create orders” maps to an operation (`operations`).

Step 2: Systematic Ambiguity Detection Through VDM-SL Conversion Attempts

This is the critical step. Attempting to convert natural language to VDM-SL exposes seven patterns of ambiguity.

First, vague quantifiers. If the specification says “a user can have multiple addresses,” VDM-SL requires choosing between `seq of Address` (no limit), `seq1 of Address` (at least one required), or `inv addrs == len addrs <= N` (maximum N). The natural language word “multiple” leaves this distinction unresolved.

Second, undefined terms. “Only active users can place orders” is written, but “active” is not defined. Defining a union type `<ACTIVE> | <SUSPENDED> | <DELETED>` in VDM-SL requires enumerating all possible states.

Third, implicit constraints. If the document simply says “users can place orders,” writing a VDM-SL pre-condition naturally raises questions: “Can a user place orders on behalf of other users?” “Is there a limit on concurrent orders?”

Fourth, missing error cases. When writing post-conditions, it becomes apparent that “what happens when payment fails” or “what if the item is out of stock” are undefined. Post-conditions cannot be written from happy-path descriptions alone.

Fifth, ambiguous relationships. If “a product is associated with a category” is stated, VDM-SL requires deciding between `map ProductId to CategoryId` (many-to-one) or `map ProductId to set of CategoryId` (many-to-many).

Sixth, temporal ambiguity. “After payment, allocate inventory” — is “after” a strict ordering? Is concurrent execution permitted? Expressing this as a chain of pre/post-conditions requires the ordering to be explicit.

Seventh, boundary conditions. If “stock has a maximum of 1000 units” is stated, whether the invariant is `<= 1000` or `< 1000` changes the meaning.

Step 3: Priority-Ordered Dialogue for Ambiguity Resolution

Detected ambiguities are classified into three priority levels and presented to the user. “Blocking” means no VDM-SL can be written without this answer (e.g., the state list of a state machine). “Important” means it affects pre-conditions, post-conditions, or invariants (e.g., inclusive vs. exclusive boundary). “Clarifying” means it would make the spec more precise but is not strictly necessary to proceed (e.g., an upper limit on a list).

Related questions are grouped to minimize dialogue rounds. Complex documents may require 3–5 rounds of clarification. This dialogue process is a concrete manifestation of the principle discussed in Section 5.3: “formal notation functions as a discipline of thought.”

A critical principle: **AI never silently resolves ambiguity.** Every interpretation choice is presented to the user. The entire purpose of formalization is to make the implicit explicit, and this principle is non-negotiable.

Step 4: VDM-SL Generation with Traceability

VDM-SL specifications are generated from the resolved information. The key here is bidirectional traceability. Each VDM-SL element carries a `[REQ-nnn]` tag and the original natural language text as a comment.

```
operations
  -- [REQ-004] "Users can create orders"
  CreateOrder(userId: UserId, items: seq1 of OrderItem) res: OrderId
    ext wr orders : map OrderId to Order
      rd users    : map UserId to User
    pre userId in set dom users
      and users(userId).status = <ACTIVE>    -- Confirmed via dialogue: active
users only
      and len items <= 50                    -- Confirmed via dialogue: max 50
items per order
    post res in set dom orders
      and orders(res).userId = userId;
```

This comment structure becomes the source information for regenerating a natural language specification in the final step.

After generation, a gap analysis is performed. “Requirements from the original document that were mapped to VDM-SL,” “requirements that could not be mapped (UI requirements, performance requirements, and other non-functional requirements),” and “elements that AI inferred” are explicitly distinguished. Inferred elements always require user confirmation.

Step 5: Mechanical Verification

The generated VDM-SL specification is mechanically verified using tools such as VDMJ. Syntax checking, type checking, and proof obligation (PO) generation are executed automatically, and results are explained in natural language. For example: “When CreateOrder returns a value of type OrderId, can you guarantee that this value is included in the keys of orders?” — presented in a form understandable without formal methods knowledge.

Issues discovered here loop back to Step 3’s dialogue. This loop ensures the specification’s internal consistency.

Step 6: Reverse Conversion from VDM-SL to Natural Language Specification

The verified VDM-SL specification is reverse-converted into a natural language specification document (Markdown) that domain experts and non-technical stakeholders can review. The conversion rules constitute a faithful translation, not a simplification. Type definitions become “what things exist.” State definitions become “what the system remembers.” Pre-conditions become “prerequisites — what must be true before the operation.” Post-conditions become “guarantees — what becomes true after a successful operation.”

The generated specification document has a structure of: overview, glossary, data model, operations, business rules, and constraints and limitations. Each element maintains traceability to the VDM-SL source. Quality checks verify completeness (every VDM-SL element appears in the document), faithfulness (the natural language accurately conveys the VDM-SL semantics), readability (no formal notation leaks into the prose), and consistency (terms are used uniformly throughout).

Critically, **this natural language specification is a derived artifact, and the VDM-SL is the source of truth.** This is stated explicitly in the document header. As a result, modification requests arising from stakeholder review flow back through the cycle of VDM-SL update → verification → regeneration, preserving specification rigor.

The overall structure of this MD→VDM-SL→MD pipeline is as follows:

```
Incomplete MD specification
  ↓ Structure analysis and requirement classification
  ↓ VDM-SL conversion attempt → Systematic ambiguity detection
  ↓ Priority-ordered dialogue (3–5 rounds)
  ↓ VDM-SL generation with traceability
  ↓ Mechanical verification (VDMJ)
  ↓ Reverse conversion to natural language specification (MD)
Verified natural language specification (VDM-SL is source of truth)
  ↓ Stakeholder review
  ↓ Modification requests → Reflected in VDM-SL (cycle restarts)
```

It was noted earlier that Phase 1 dialogue is iterative. This pipeline provides the concrete mechanism for that iteration. Domain experts’ tacit knowledge cannot be fully elicited in a single dialogue session — some rules only surface when the system is actually used. The review cycle mediated by natural language specifications enables tacit knowledge to be progressively formalized.

Phase 2: AI-Driven Technology Selection and Architecture — Integrating Non-Functional Requirements

The formal specification from Phase 1 defines *what* to compute, but not at what scale, speed, or cost. Phase 2 integrates **non-functional requirements** provided separately by the human, combining them with the formal specification to make technical design decisions.

For the sorting specification above, suppose the human states: “Data scale is approximately $n = 10^{30}$,” “Compute resources and budget are limited,” and “Stable sorting is not required.” Combining these constraints with the specification, AI determines:

- Programming language (weighing type safety, performance requirements, ecosystem)
- Algorithm selection ($n = 10^{30}$ requires external sorting; stability not required favors quicksort variants)
- Frameworks and libraries
- Architecture (whether distributed processing is needed, microservices vs. monolith, database selection, etc.)
- Infrastructure (server specifications, cloud provider, container orchestration, etc.)

The key structural insight: the formal specification provides the **correctness criterion** while non-functional requirements provide the **design constraints**, and the two are independent. Bubble sort satisfies the VDM-SL specification but is $O(n^2)$ —unworkable for $n = 10^{30}$ —and is rejected based on non-functional requirements. The separation of formal specification and non-functional requirements prevents correctness and efficiency from becoming entangled.

Eliciting Non-Functional Requirements Through Dialogue: A practical challenge arises here. Domain experts may not be able to specify quantitative non-functional requirements such as “response time under 200ms” from the outset. Whether 200ms or 500ms is appropriate involves trade-offs between user experience and infrastructure cost—a domain distinct from pure business knowledge.

This methodology addresses this through **experience-based elicitation** by AI. For example, AI generates a mock UI and intentionally reproduces different response delays (100ms, 300ms, 500ms, 1000ms) for the human to interact with. The human can then judge experientially: “this feels too slow” or “this is acceptable.” Alternatively, AI presents rough infrastructure cost estimates: “achieving 200ms requires this configuration at \$X/month; 500ms requires this configuration at \$Y/month,” enabling the domain expert to make a business decision. The quantification of non-functional requirements is thus resolved within the same dialogue structure as Phase 1 — AI proposes, human decides.

Phase 3: AI Code Generation

AI generates code corresponding to the VDM-SL specification. The key points:

- VDM-SL pre-conditions are implemented as runtime validation or assertions
- Post-conditions serve as correctness criteria for the generated code
- Invariants become design constraints on data structures
- Type definitions are mapped as directly as possible to the implementation language’s type system

A notable aspect of this phase is the design decision involved in translating VDM-SL's abstract types into concrete data structures. VDM-SL is intentionally abstract about implementation details. For example, `seq of T` (an ordered collection) might become an `Array` if random access dominates, a `LinkedList` if head insertions and deletions are frequent, or an `ArrayList` or `Deque` if both are needed. Similarly, `set of T` could map to a `HashSet`, `TreeSet`, or `BitSet`; `map T1 to T2` could become a `HashMap`, `TreeMap`, or `ConcurrentHashMap`.

These decisions cannot be derived from type definitions alone. AI makes them by synthesizing the operation frequency patterns in the specification (is lookup dominant, or insertion?), the non-functional requirements established in Phase 2 (concurrency, memory constraints, latency requirements), and the characteristics of the selected language and frameworks. In other words, translating from the specification's "What" to the implementation's "How" is not a trivial mapping but an optimization process coupled with Phase 2's architectural decisions.

Phase 4: Specification Conformance Verification

The generated code is verified against the formal specification. This is the first point at which testing appears—but its role is fundamentally different from TDD. Tests are property-based tests derived automatically from the specification. No human writes individual test cases by hand.

Because the specification is precisely defined, test generation enjoys enormous flexibility. Random tests that generate inputs satisfying pre-conditions and verify post-conditions. Boundary-value tests that probe the edges of invariant constraints. Systematic tests that cover all state-transition paths. All of these can be automatically generated from the specification. For the sorting example, the two post-conditions—`bagOf(1) = bagOf(RESULT)` and the ordering condition—generate tests with empty sequences, single elements, all-identical elements, reverse-ordered inputs, and massive sequences, all without human intervention.

Phases 2–4 Are an Autonomous AI Cycle

A critical point deserves emphasis: the cycle of Phase 2 (design) → Phase 3 (implementation) → Phase 4 (verification) is **executed entirely by AI, autonomously**. If Phase 4 detects a specification violation, AI either fixes the implementation or revisits design decisions (algorithm selection, data structure choices) and re-implements. No human intervenes in this feedback loop.

This is possible precisely because the specification is formally defined. If the specification were ambiguous, AI could not autonomously determine whether a defect is a specification problem or an implementation problem. But a formal specification provides an unambiguous criterion for correctness, enabling AI to automatically judge "this does not satisfy the specification → correction is needed." Once the human confirms the specification in Phase 1, AI can be entrusted with the entire process until a working system is delivered.

The Specification-Centric Short Cycle—A Third Structure, Neither Waterfall Nor Agile

Let us clarify the cycle structure of this methodology by contrasting it with conventional approaches.

In waterfall, the flow is linear: requirements → specification → design → coding → release → requirements. In agile, the cycle is: requirements → lightweight design → coding → release → requirements—improvement driven by post-release feedback. In both cases, tacit knowledge surfaces **after release**—that is, only after humans use a working system.

This methodology's structure is fundamentally different. Because the formal specification is rigorous, **specification-level testing and mock generation become possible without waiting for release**. The cycle is as follows:

Requirements articulation ↔ Formal specification ↔ Spec testing and mock presentation → Release

The human conveys requirements; AI generates a VDM-SL specification. AI verifies the specification's internal consistency and presents contradictions or undefined edge cases to the human. Furthermore, AI generates mocks (prototypes) automatically from the specification and has the human interact with them, eliciting “tacit knowledge that only surfaces through hands-on experience”—**before a single line of code is written**. Specification testing surfaces logical contradictions; mocks surface experiential insights. This dual feedback loop operates at the specification level.

In the building analogy, this is like walking through the floor plan in a 3D model before construction—discovering “this traffic flow is awkward” or “I want a shelf here” through VR previewing. Far more efficient than renovating after moving in.

Once this cycle has run sufficiently, Phases 2–4 are executed autonomously by AI, delivering a system that conforms to the specification. Of course, the possibility of new tacit knowledge surfacing after release does not drop to zero. However, having passed through both logical verification at the specification level and experiential verification through mocks, the volume of tacit knowledge that emerges post-release is **drastically reduced** compared to agile methods. And should something be discovered, the team simply returns to the same specification-centric cycle. There is no need to rewrite tests or worry about refactoring side effects as in agile—updating the specification causes AI to generate a new system.

The fact that the cycle is centered on the specification—this is the structural hallmark of this methodology.

4.2 What Humans Need to Know

The skills required of humans in this paradigm are fundamentally different from traditional development.

Required skills:

- **Domain knowledge:** Deep understanding of the business domain being developed. This is the irreplaceable human contribution—the one AI cannot substitute. Decisions like “should deactivated users’ email addresses be reusable?” can only be made by someone who understands the business context.
- **Logical dialogue ability:** The ability to identify logical contradictions or missing requirements through natural-language conversation when AI explains the specification. Reading formal notation is not required, but being able to reason in plain language about structures like “if A then B” or “for every X, Y holds” is valuable.
- **Ability to articulate requirements:** The ability to make implicit domain knowledge explicit in a form that AI can process. This is the same skill as traditional requirements definition, but the bar for clarity rises when conversing with AI—ambiguity cannot be left unresolved.

Skills that become unnecessary:

- Reading or writing formal method notation (AI generates the specification and explains it in plain language)
- Proficiency in specific programming languages (AI selects the appropriate language and writes the code)
- Detailed knowledge of frameworks and libraries (AI selects the optimal ones)
- Test case design (automatically derived from the specification)
- Detailed infrastructure design (AI derives this from non-functional requirements in the specification)

4.3 Comparison with Traditional Development

Dimension	TDD	Formal Methods + AI-Driven
Correctness guarantee	Inductive (finite test cases)	Deductive (logical specification)
Specification ambiguity	Tests constitute an implicit spec	Specification is explicit and precise
Design driver	Test writability	Logical structure of requirements
Human role	Coder and tester	Client (domain expert + decision-maker)
AI utilization	Code completion	Design → implementation → verification, end-to-end
Scalability	Test count grows explosively	Specification scales with problem complexity
Bug detection timing	After implementation (at test runtime)	During specification (detected as logical contradiction)

Chapter 5: Why Formal Methods—Why Now?

5.1 The Three Walls That Blocked Formal Methods Before LLMs

Formal methods have existed since the 1970s yet never achieved broad industrial adoption. Three walls stood in the way.

First, writing formal specifications required advanced mathematical training. People capable of writing VDM-SL or Z notation were scarce, and demanding that from an entire team was unrealistic.

Second, a large gap existed between formal specifications and working code. Even a beautiful specification required manual effort to implement, and bugs crept in at that translation step.

Third, the return on investment was opaque. The time cost of writing specifications was high; the payoff was unclear; adoption was confined to safety-critical domains such as aerospace, rail, and medical devices.

5.2 Why LLMs Are a Game Changer

LLMs dissolve all three walls.

Wall 1 dissolved: Humans no longer need to write or read formal specifications. Business requirements are expressed in natural language; AI converts them to VDM-SL. AI explains the specification in natural language; humans evaluate the explanation. A building client grasps the design through dialogue with the architect—without reading a single structural calculation.

Wall 2 dissolved: AI performs the translation from formal specification to code. The gap between specification and implementation shrinks dramatically. AI generates code with an understanding of pre-conditions, post-conditions, and invariants—reducing the risk of bugs entering at the translation step. That said, current LLMs can hallucinate when handling complex specifications, so conformance verification (Phase 4) cannot be skipped.

Wall 3 dissolved: AI automation has slashed the cost of specification. What once took weeks can now be accomplished in hours or days through AI dialogue. The ROI equation has changed fundamentally.

5.3 Rebutting “If Only AI Reads It, Why Bother with Formal Methods?”

A natural objection arises: if humans don’t read the formal specification, why write it in VDM-SL at all? Couldn’t AI manage the specification in natural language?

The answer is clear: **formal notation functions as a discipline of thought for AI, not just for humans.**

First, formal notation structurally eliminates ambiguity. Writing “a user has one email address” in natural language leaves open whether this means exactly one, at least one, or at most one. Defining `email: Email` in VDM-SL settles it by notation alone. When AI manages specifications in natural language, it risks introducing exactly this kind of ambiguity. Formal notation eliminates that risk by construction.

Second, formal specifications are mechanically verifiable intermediate artifacts. VDM-SL specifications can be type-checked and consistency-checked using tools such as Overture Tool. Natural-language specification documents cannot. Even when the AI that wrote the specification and the AI that implements it are different sessions or different models, the formal specification binds both as a precise contract.

Third, there is the question of auditability. Formal specifications can be verified after the fact by other AI systems, other tools, or future verification systems—regardless of whether any human reads them. For example, a security audit can mechanically confirm from the formal specification whether all operations are accessible only to authenticated users. A compliance review can automatically verify whether a personal data deletion operation propagates to all related tables. Performing equivalent mechanical verification on natural-language specifications is fundamentally difficult due to inherent ambiguity. This auditability value grows as project scale increases.

In short, formal methods exist not “for humans” but “for logical rigor.” Writing in formal notation improves AI’s reasoning precision even when no human will ever read the result. This is the same principle by which mathematicians reach more accurate conclusions using symbols than intuition alone.

5.4 The Transition Strategy Before AGI

Current LLMs are not AGI. They lack the ability to autonomously understand business requirements and design optimal systems from scratch. But they already demonstrate practical capability in understanding formal specifications and generating conformant code.

This profile—strong at specification understanding and implementation, limited at requirements definition—is ideal for combination with formal methods. Humans convey the business “What” in natural language; AI translates it into a formal specification, verifies it through dialogue with the human, and then implements the technical “How.”

When AGI arrives, AI may be able to handle even the “What.” Until then, formal methods combined with AI-driven development is the most rational approach available.

5.5 Interim Operational Measures—Bridging Ideal and Today

This methodology’s concept targets the optimization of AI-driven development **in the period from the near future—when AI agent autonomy has advanced further—through AGI arrival.** The ideal workflow (fully autonomous Phase 2–4 cycles) cannot be fully realized for complex systems with current LLMs. Acknowledging this honestly, we present **interim measures for gradually adopting this methodology starting today.**

Phase 2 (Design) Interim Measures: Current LLMs can autonomously handle design for simple CRUD systems and API services. However, for complex distributed systems or systems with advanced security requirements, intervention by an experienced architect may still be needed. As an interim measure, AI generates design proposals and a human architect reviews and corrects them—a hybrid model. The key is to position this architect involvement as **transitional**, with the explicit expectation that it will be progressively reduced as AI autonomy improves.

Phase 4 (Verification) Interim Measures: The cycle of automatically generating property-based tests from VDM-SL post-conditions and executing them can currently run fully autonomously only for simple specifications. For specifications involving complex state transitions or concurrent processing, QA engineer expertise may be needed for test strategy design. In this case too, the QA engineer does not write test cases by hand but instead reviews and adjusts the direction of AI-generated test strategies. Test execution and result evaluation remain AI-autonomous.

Common Principle Across Interim Measures: In all interim measures, human intervention is limited to “reviewing AI output and correcting direction.” Humans do not write code from scratch or handcraft test cases. This ensures a natural reduction of human intervention points as AI autonomy improves. The interim measures do not negate the methodology’s ideal—they are a graduated migration path toward it.

Chapter 6: A Practical Roadmap

6.1 Individual Level

Step 1: Practice logical articulation (1–2 weeks)

Start by articulating familiar business rules precisely in natural language. For an e-commerce order process: “Items with zero inventory cannot be ordered.” “A user can have at most one order in processing at a time.” Write these as explicit conditional statements. Understanding concepts like sets, maps, and universal/existential quantification at the natural-language level is sufficient. There is no need to learn VDM-SL notation.

Step 2: Practice specification dialogue with AI (1–2 weeks)

Ask AI to “write a formal VDM-SL specification for a user management feature.” As AI explains the generated specification, practice identifying gaps and contradictions. The goal is to develop the ability to refine a specification by asking questions like “What happens to a deactivated user’s data?” or “Can an administrator delete their own account?”

Step 3: Run the full cycle on a small project (2–4 weeks)

Execute the entire pipeline—from specification dialogue through code generation to reviewing the deliverable. Start with a small API with basic CRUD operations. The essential experience is the feedback loop: use the generated artifact, find behaviors that differ from what was agreed in the specification dialogue, and return that feedback to AI.

6.2 Team and Organizational Level

For organizational adoption, begin with a single small new service as a pilot project built entirely with formal methods + AI-driven development. Do not attempt to migrate existing large systems immediately. Demonstrate value first, then scale.

The review process changes fundamentally. Traditional code review is replaced by **specification dialogue review**. Share the AI specification-dialogue logs; convene domain-knowledgeable humans to discuss whether the specification correctly reflects the requirements. Delegate implementation details to AI; focus human attention at the specification level.

Chapter 7: Honest Limitations

7.1 Where Formal Methods Are Not Enough

Formal methods are not universal. Complementary approaches remain necessary in several areas.

UI/UX specification: The “usability” and “aesthetics” of user interfaces are difficult to formalize. Prototyping and user testing remain valuable. However, the business logic beneath the UI—state transitions, validation rules, and so on—is fully formalizable.

Performance characteristics: VDM-SL describes *what* is computed, not *how fast*. Performance requirements must be handled separately as non-functional requirements.

External system integration: The actual behavior of third-party APIs cannot be fully captured in a formal specification. Integration testing remains valuable at system boundaries.

7.2 Tests Do Not Disappear Entirely

The article’s title is provocative, but it does not wholesale reject testing. What it rejects is the TDD *paradigm*—the idea that tests should be the center of design and quality assurance.

Testing persists in this approach in the following roles:

- Property-based tests automatically derived from the specification (conformance verification)
- Integration tests for external system boundaries
- Performance and load tests
- End-to-end / acceptance tests for UI

The shift is that testing moves from center stage to a supporting role.

7.3 Multi-Agent Coordination and Applicable Scale — Scaling Through Formal Specifications

Can a single AI agent autonomously build an entire large-scale system? No—context window constraints prevent it from grasping a system of tens of thousands of lines at once. But this is the wrong question. In human development teams, no single engineer holds the entire system in their head either. Team development works because interfaces between modules have been agreed upon.

Formal specifications provide precisely this “interface agreement that enables team development”—rigorously. And this is the key that makes autonomous construction of medium-to-large systems possible even with current AI agents.

Multi-Agent Architecture with Formal Specifications as Contracts

The structure is as follows.

In Phase 1 (human + architect AI), the formal specification for the entire system is defined. The core activity here is module decomposition and the definition of inter-module interface specifications—the pre-conditions and post-conditions of operations each module exposes. This set of specifications becomes the “contract” for all agents.

In Phases 2–4, independent AI agents work on their assigned modules **in parallel**, each executing design, implementation, and verification autonomously. Agent A handles the inventory management module, Agent B handles order processing, Agent C handles payment. Each agent’s context needs only “its own module’s specification” plus “the interface specifications of modules it depends on (pre/post-conditions only)” —implementation details of other modules are entirely unnecessary. This fits comfortably within current context windows.

For integration verification, a dedicated integration agent cross-checks the published interface specifications of all modules, mechanically verifying the $A.post \Rightarrow B.pre$ compositional consistency. This agent need not examine implementation code at all—it specializes in specification-level consistency checking.

Why this structure is impossible with natural-language specifications but possible with formal specifications comes down to three reasons. First, each agent can independently determine whether its implementation satisfies its spec. Because the correctness criterion is unambiguous, no “alignment meetings” with other agents are needed. Second, inter-module consistency verification can be performed mechanically. Implication relationships between post-conditions and pre-conditions are tool-verifiable. Third, the context each agent must hold is minimized. No agent needs to know the internals of other modules—only their interface specifications.

This is the same principle by which API documentation and interface definitions (OpenAPI, Protocol Buffers, etc.) enable division of labor in human teams. However, natural-language API documentation leaves ambiguities such as “only success responses documented, error cases undefined.” Formal specifications structurally eliminate this ambiguity.

Remaining Challenges and the Human Role

Several challenges remain at this point: unifying cross-cutting concerns (authentication/authorization, logging, error handling patterns), managing shared data model consistency (database schemas), and automating agent orchestration (execution order and dependency control). These are not problems of fundamental impossibility but engineering challenges—their resolution is a matter of time.

These cross-cutting design decisions are resolved autonomously by AI in Phase 2, then included in each agent’s instructions. The human’s role is strictly that of a **domain expert**—defining what to build, not how to build it. In the building analogy, the homeowner says “I want a bright living room that’s warm in winter” and “my budget is X.” The sequencing of foundation, framing, elec-

trical, and plumbing—and which contractors to hire—is the builder’s domain, not the homeowner’s. Similarly, technology selection (language, framework, database) and architecture design are AI’s autonomous responsibility.

Scaling Outlook

The combination of multi-agent coordination and formal specifications makes medium-scale systems (tens of thousands of lines) theoretically achievable even with current AI models; the maturation of the orchestration layer is the key to practical deployment. Furthermore, as context windows expand, inter-agent coordination protocols become standardized, and long-term memory improves, large-scale systems (over one hundred thousand lines) come into view. Crucially, across all of these technological advances, formal specifications function as “rigorous contracts” between agents. With natural-language specifications, coordination ambiguity grows exponentially as the number of agents increases; with formal specifications, module boundary consistency remains mechanically verifiable regardless of agent count—making the approach inherently scalable.

Conclusion: The Human Role in AI Agent-Driven Development

The multi-agent paradigm of formal methods + AI-driven development fundamentally redefines the human role in software development. The developer who was “a person who writes code” becomes a **domain expert**—“a person who defines what to build and why, and evaluates whether the AI’s proposals match their business intent.”

This is not a devaluation of human contribution—it is an **elevation in abstraction level**. It is the same evolution as the transition from assembly language to high-level languages, from manual memory management to garbage collection. Humans become free to concentrate on the most fundamental questions: “What should be built?” and “Why?”—while technology selection, system architecture, and implementation are delegated to AI.

What is required is deep understanding of one’s own business domain and the ability to engage in logical dialogue with AI. Reading or writing formal notation is not required—that is the AI agents’ responsibility. The human’s role is to exercise domain expertise when the architect AI explains in natural language: “Is this module decomposition appropriate for the requirements?” Whether the system ends up written in Python, Ruby, or TypeScript is not the human’s concern—that decision is made autonomously by AI based on the formal specification and non-functional requirements. The quality of domain judgment is the core competence of humans in the multi-agent development era.

Formal methods function as a discipline of each AI agent's reasoning, as rigorous contracts between agents, as a means of mechanical verification at module boundaries, and as an auditable record for the future. Humans need not read them—but writing formally carries decisive value, especially in contexts where multiple agents coordinate. This is the central claim of this article.

The era of TDD's dominance is drawing to a close. We are witnessing the dawn of multi-agent AI-driven development, where formal specifications serve as “contracts” between agents. And the role humans must play in this new paradigm is not writing code, but providing the **decision-making and oversight** that ensures the AI agent team builds the right thing, correctly.

References

- Dijkstra, E.W. (1969). *Notes on Structured Programming*.
- Jones, C.B. (1990). *Systematic Software Development using VDM*. Prentice Hall.
- Fitzgerald, J. & Larsen, P.G. (2009). *Modelling Systems: Practical Tools and Techniques in Software Development*. Cambridge University Press.
- Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.
- Newcombe, C. et al. (2015). “How Amazon Web Services Uses Formal Methods.” *Communications of the ACM*, 58(4).
- Jackson, D. (2012). *Software Abstractions: Logic, Language, and Analysis*. MIT Press.
- Overture Tool Project: <https://www.overturetool.org/>
- Larsen, P.G. et al. (2010). “Industrial Applications of VDM.” In *Bentley Historical Library*.
- Bicarregui, J. et al. (2009). “Proof and Model Checking for Protocol Design.” *Formal Aspects of Computing*, 21(1-2).